

Day 19 — full classification portfolio project

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 3, Day 5 — complete line-by-line companion to `day19_portfolio_project.py`

Same format as the Day 17 and Day 18 lessons: every section explains the idea first, then shows the exact code, then walks through what each line does and why it's written that way. This script pulls Days 15-18 into one finished pipeline that ends with an actual reloadable model file, not just printed numbers.

0. Imports and setup

Days 15-18 each practiced one isolated skill on the Titanic dataset. Day 19 is the day those skills get assembled into a single pipeline that a real job would actually want: pick the best model honestly, pick a threshold without cheating by looking at the test set, evaluate once, and save something that could be deployed.

```
import os
import urllib.request

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_predict
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    roc_curve, roc_auc_score, precision_recall_curve,
    confusion_matrix, classification_report, accuracy_score
)
```

→ `joblib` — new this script. The standard scikit-learn-recommended library for saving a fitted model to disk and loading it back later without retraining

→ `cross_val_predict` — new this script. Unlike `cross_val_score` (Day 17), which only returns a score per fold, this returns an actual prediction for every row, produced by whichever fold held that row out — used here to get honest probabilities on the full training set without ever fitting on a row and predicting on that same row

→ `confusion_matrix`, `classification_report` — new this script. `classification_report` prints precision, recall, and F1 for both classes in one formatted block instead of computing each metric separately like Day 18 did; `confusion_matrix` shows the raw counts those metrics are built from

→ all three model classes from Day 16-17 (`LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`) are imported together, since this script now searches across all three instead of committing to one upfront

1. prepare_data() — same pipeline, one new return value

Identical cleaning and encoding to Day 15-18: median-fill Age, drop Cabin, mode-fill Embarked, engineer FamilySize, one-hot encode Sex and Embarked, stratified 80/20 split. The one addition is returning the list of final feature column names, which step 4 needs later to save alongside the model.

```
def prepare_data(path=DATA_PATH, url=DATA_URL):
    if not os.path.exists(path):
        urllib.request.urlretrieve(url, path)

    df = pd.read_csv(path)
    df["Age"] = df["Age"].fillna(df["Age"].median())
    df = df.drop(columns=["Cabin"])
    df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
    df["FamilySize"] = df["SibSp"] + df["Parch"] + 1

    y = df["Survived"]
    feature_cols = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare",
                    "Embarked", "FamilySize"]
    X = pd.get_dummies(df[feature_cols], columns=["Sex", "Embarked"], drop_first=True)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    return X_train, X_test, y_train, y_test, list(X.columns)
```

→ `list(X.columns)` — converts the pandas column index into a plain Python list of strings before returning it. This exact ordered list of names is what gets bundled into the saved model file in step 4, so anyone reloading the model later knows precisely which columns to feed it and in what order

→ everything else in this function is byte-for-byte identical to Day 15-18's `prepare_data()`, because the whole point of finalizing a pipeline is that the data preparation stops changing while the modeling decisions around it get more sophisticated

2. select_and_tune_model() — searching 3 models at once

Day 17 tuned one random forest with `GridSearchCV`. Day 19 runs a separate `GridSearchCV` for each of three model types, then compares their best cross-validated scores head to head. Whichever model's best tuned version scores highest becomes the final answer — the model family itself is now a decision made by evidence, not assumption.

```

def select_and_tune_model(X_train, y_train):
    candidates = {
        "Logistic Regression": (
            LogisticRegression(max_iter=1000),
            {"C": [0.1, 1, 10]}),
        "Decision Tree": (
            DecisionTreeClassifier(random_state=42),
            {"max_depth": [3, 5, None], "min_samples_leaf": [5, 10]}),
        "Random Forest": (
            RandomForestClassifier(random_state=42),
            {"n_estimators": [100, 200], "max_depth": [4, 8]}),
    }

    best_name, best_model, best_score = None, None, -1
    for name, (estimator, grid) in candidates.items():
        search = GridSearchCV(estimator, grid, cv=5, n_jobs=-1)
        search.fit(X_train, y_train)
        if search.best_score_ > best_score:
            best_name, best_model, best_score = name, search.best_estimator_, search.best_score_

    return best_name, best_model, best_score

```

→ `candidates` — a dictionary where each value is a tuple of (unfitted model instance, its own hyperparameter grid). Each model gets a different grid because they have different hyperparameters — logistic regression tunes `C` (regularization strength), trees tune depth and leaf size, forests tune both tree count and depth

→ `"C": [0.1, 1, 10]` — `C` controls how strongly logistic regression is penalized for large coefficients; smaller `C` means stronger regularization (simpler, more conservative model), larger `C` means the model is allowed to fit the training data more closely

→ `best_name, best_model, best_score = None, None, -1` — initializes tracking variables before the loop; `-1` is used instead of `0` because a real accuracy score can never be negative, guaranteeing the very first candidate always replaces this initial placeholder

→ `for name, (estimator, grid) in candidates.items()` — nested unpacking: `.items()` gives (key, value) pairs, and since each value is itself a 2-tuple, `(estimator, grid)` unpacks it further in the same line

→ `search.best_estimator_` — not just the best hyperparameters, but an actual already-fitted model object using those hyperparameters, ready to be compared or reused directly without rebuilding it from the parameter dictionary

→ the `if search.best_score_ > best_score` comparison inside the loop is what makes this a true 3-way competition — each iteration only overwrites the running winner if the new candidate's tuned score is strictly better

3. `choose_threshold()` — picking a cutoff without touching the test set

Day 18 picked an F1-optimal threshold using test-set probabilities, which works for a one-off lesson but is a habit worth breaking. If you tune your threshold using the same data you later report performance on, that performance number is quietly optimistic. `cross_val_predict` solves this: it produces a probability for every training row using only folds that row wasn't trained on, so the threshold can be chosen honestly before the test set is ever opened.

```
def choose_threshold(best_model, X_train, y_train):
    oof_probs = cross_val_predict(
        best_model, X_train, y_train, cv=5, method="predict_proba"
    )[:, 1]

    precision, recall, thresholds = precision_recall_curve(y_train, oof_probs)
    denom = precision[:-1] + recall[:-1]
    numer = 2 * precision[:-1] * recall[:-1]
    f1_scores = np.divide(numer, denom, out=np.zeros_like(numer), where=denom > 0)

    best_idx = np.argmax(f1_scores)
    best_threshold = thresholds[best_idx]

    return best_threshold
```

→ `cross_val_predict(best_model, X_train, y_train, cv=5, method="predict_proba")` — internally does the same 5-fold split as `cross_val_score`, but instead of scoring each fold it collects the held-out predictions from every fold and stitches them back together into one array the same length as `X_train` — "oof" stands for out-of-fold, meaning every value came from a model that never saw that particular row during training

→ `method="predict_proba"` — tells `cross_val_predict` to collect probability arrays instead of hard 0/1 labels, which is why `{C}[:, 1]{E}` is needed afterward to keep only the survival-probability column, same pattern as Day 18

→ `precision_recall_curve(y_train, oof_probs)` — notice this is called on `y_train`, not `y_test`. The entire threshold search in this function never references `X_test` or `y_test` anywhere

→ the F1-sweep logic (the safe-division block and `np.argmax`) is identical to Day 18's `compute_pr_curve()` — same technique, just applied to out-of-fold training probabilities instead of test probabilities

4. `final_evaluation()` — the one and only look at the test set

Every decision up to this point — which model, which hyperparameters, which threshold — was made without `X_test` or `y_test`. This function is where the test set finally gets used, exactly once, to report an honest, unpeeked-at estimate of real-world performance.

```
def final_evaluation(best_model, best_threshold, X_train, y_train, X_test, y_test):
    best_model.fit(X_train, y_train)
    test_probs = best_model.predict_proba(X_test)[: , 1]
    test_preds = (test_probs >= best_threshold).astype(int)

    cm = confusion_matrix(y_test, test_preds)

    print(classification_report(y_test, test_preds, target_names=["died", "survived"]))

    fpr, tpr, _ = roc_curve(y_test, test_probs)
    precision, recall, _ = precision_recall_curve(y_test, test_probs)
    plt.savefig("day19_final_evaluation.png", dpi=150)

    return best_model
```

→ `best_model.fit(X_train, y_train)` — refits on the entire training set. During `GridSearchCV`, `best_estimator_` was actually fit on a specific CV split internally; refitting here on the full `X_train` ensures the final model has learned from every available training row before its one real test

→ `test_preds = (test_probs >= best_threshold).astype(int)` — applies the threshold chosen in step 3 (from training data alone) to brand-new probabilities computed on the test set. The threshold itself was never influenced by these particular numbers

→ `confusion_matrix(y_test, test_preds)` — returns a 2×2 array: `cm[0][0]` is true negatives (correctly predicted died), `cm[0][1]` is false positives, `cm[1][0]` is false negatives, `cm[1][1]` is true positives. The script indexes into this array directly to print a labeled table

→ `classification_report(..., target_names=["died", "survived"])` — computes precision, recall, and F1 separately for each class and prints them as one formatted block, which is faster than calling `precision_score/recall_score/f1_score` three times each like Day 18 did in its threshold-sweep table

→ the function returns `best_model` (now fitted) so step 5 can save this exact fitted object, rather than an unfitted copy from the candidates dictionary

5. `save_model()` — bundling model, threshold, and columns together

A fitted scikit-learn model is just a Python object, and `joblib` can pickle any Python object to disk. But saving only the raw model isn't enough to make it usable later: whoever reloads it also needs to know the decision threshold and the exact feature columns it expects, in the right order. Bundling all three into one dictionary means there's only one file to keep track of, and no risk of the threshold getting separated from the model it belongs to.

```
def save_model(model, threshold, feature_columns, path="day19_final_model.joblib"):
    bundle = {
        "model": model,
        "threshold": threshold,
        "feature_columns": feature_columns,
    }
    joblib.dump(bundle, path)
```

- `bundle = {...}` — a single Python dictionary holding all three pieces of information a future user of this model needs, not just the model itself
- `joblib.dump(bundle, path)` — serializes the entire dictionary (including the trained model object inside it) to a binary file at `{C}path{E}`. `joblib` is preferred over Python's built-in `pickle` for `scikit-learn` models because it handles NumPy arrays inside the model more efficiently
- `path="day19_final_model.joblib"` — given a default value directly in the function signature, so it can be called as `{C}save_model(model, threshold, cols){E}` normally, but overridden with a different filename later without changing the function

6. `verify_reload()` — proving the saved file actually works

Saving a file proves nothing by itself — the file could be corrupted, could be missing something needed to reload correctly, or the load path could silently produce a different object. This function reloads the file inside the same script run and directly compares its predictions against the original in-memory model, the same instinct as testing a backup by actually restoring it rather than trusting it exists.

```
def verify_reload(original_model, threshold, X_test, y_test, path="day19_final_model.joblib"):
    bundle = joblib.load(path)
    reloaded_model = bundle["model"]
    reloaded_threshold = bundle["threshold"]

    sample = X_test.iloc[:10]
    original_probs = original_model.predict_proba(sample)[: , 1]
    reloaded_probs = reloaded_model.predict_proba(sample)[: , 1]

    original_preds = (original_probs >= threshold).astype(int)
    reloaded_preds = (reloaded_probs >= reloaded_threshold).astype(int)

    match = np.array_equal(original_preds, reloaded_preds)
```

- `joblib.load(path)` — reads the binary file back and reconstructs the exact same dictionary that was saved, including a fully working fitted model object
- `bundle["model"], bundle["threshold"]` — unpacks the reloaded dictionary using the same string keys it was saved with in step 5. This only works because those key names match exactly between `save_model()` and `verify_reload()`
- `X_test.iloc[:10]` — takes just the first 10 test rows for a quick sanity check rather than re-running the full evaluation; the goal here is proving the file works, not re-measuring accuracy
- `np.array_equal(original_preds, reloaded_preds)` — compares two NumPy arrays element by element and returns a single True/False, rather than comparing them with `==` directly, which would return an array of booleans instead of one combined answer
- if this check ever printed False, it would mean something about the saved file doesn't faithfully reproduce the original model — a real bug worth catching before handing the file to anyone else

7. `main()` — the call order is the whole discipline

```
def main():
    X_train, X_test, y_train, y_test, feature_columns = prepare_data()

    best_name, best_model, best_cv_score = select_and_tune_model(X_train, y_train)
    best_threshold = choose_threshold(best_model, X_train, y_train)
    fitted_model = final_evaluation(best_model, best_threshold, X_train, y_train, X_test,
    save_model(fitted_model, best_threshold, feature_columns)
    verify_reload(fitted_model, best_threshold, X_test, y_test)
```

→ notice `X_test` and `y_test` are created in the very first line but not passed into a function until `final_evaluation()` — `select_and_tune_model()` and `choose_threshold()` are both called with `X_train/y_train` only, which is what actually enforces "no test-set peeking" rather than just claiming it in a comment

→ `fitted_model` returned from `final_evaluation()` (already `.fit()` on the full training set) is what gets passed to both `save_model()` and `verify_reload()` — the unfitted or partially-fitted versions from earlier steps are never saved

8. Deliverable and push

```
git add day19_portfolio_project.py day19_final_evaluation.png day19_final_model.joblib
git commit -m "Day 19: full classification portfolio - model selection, tuning, thresholdi
git push
```